

Apache Hadoop: A Critical Analysis and Comparison with Apache Spark in Distributed Data Processing

SABRINA HENG WEI QI
Faculty of Computing
Universiti Teknologi Malaysia
Johor, Malaysia
wei04@graduate.utm.my

Abstract— The rapid growth of data in recent years has created new challenges for traditional data processing systems, particularly in terms of scalability and efficiency. To address these challenges, distributed computing frameworks such as Apache Hadoop have been widely adopted. Hadoop provides a reliable and scalable approach to managing large datasets by distributing storage and computation across multiple machines. However, as data processing requirements continue to evolve, especially with the increasing demand for faster and more flexible analytics, certain limitations of Hadoop have become more apparent. This study examines the architecture and performance of Apache Hadoop and explores its capabilities in handling large-scale data processing tasks. In addition, Apache Spark is introduced as a comparative framework to provide a broader perspective on alternative processing approaches. By analyzing the differences between disk-based and in-memory computation, this paper aims to offer a clearer understanding of the strengths, limitations, and trade-offs involved in modern distributed data processing systems.

Keywords—*Apache Hadoop, Apache Spark, Distributed Computing, Big Data Processing, MapReduce*

I. INTRODUCTION

The amount of data generated in today's digital world has grown at an unprecedented rate. In social media communication to business dealings, organisations are generating vast amounts of data that must be stored and analysed effectively. Such scale can no longer be managed using conventional data processing systems which are usually based on one machine. This has contributed to the growing significance of distributed computing systems which are capable of processing data across a number of machines

One of the first solutions to this issue was created in the form of Apache Hadoop. It enables storage and processing of large datasets across groups of computers, enabling data to be processed at a significantly larger scale than previously. Hadoop provides a dependable and fault resistant means of handling big data with features like the Hadoop Distributed File System (HDFS) and the MapReduce programming model [1]. Due to these features, it has been extensively applied in industry in batch processing operations.

But with the changing requirements of data processing to real time interactive analytics, there have been concerns whether Hadoop is still relevant to the current applications. Its dependency on disk-based processing can result in slower performance, especially in the case of iterative workloads. To address these issues, more recent systems like Apache Spark have been added, providing different solutions to distributed data processing [2].

This paper discusses Apache Hadoop in depth, its functionality, its strengths and weaknesses. It also contrasts Hadoop with Apache Spark to gain a better insight into the development of distributed data processing and what it entails as far as the present and future applications are concerned.

II. APACHE HADOOP OVERVIEW

Apache Hadoop is an open-source framework that is developed to process and store large amounts of data by distributing storage and processing over multiple computers. Hadoop does not only rely on one single system, it divides data into small units and runs them simultaneously, which able to enhance efficiency and scalability.

A. Hadoop Architecture

To understand how storage and computation are managed, the underlying physical and logical layout of the cluster is illustrated in Figure 1.

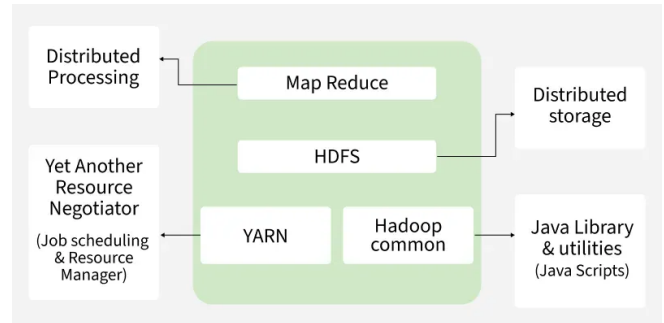


Figure 1: Hadoop Architecture [8]

- **Hadoop Distributed File System (HDFS)**
Hadoop Distributed File System is in charge of the storage of data in cluster of nodes. Big files are broken down into small blocks and stored in more than one machine. In order to achieve reliability, every block is replicated, such that the system can still operate even when some of the nodes fail [3].
- **MapReduce**
The fundamental processing model in Hadoop is MapReduce. It breaks down tasks into two phases, the map phase where data is broken down into smaller units and the reduce phase where the results are synthesized. The method enables processing of large datasets in a cost-effective manner in parallel [1].

- **YARN (Yet Another Resource Negotiator)**
YARN is a system resource management and coordinates the distribution of tasks in the cluster. It makes sure that computing resources are utilized in an efficient manner [4].
- **Hadoop Common (Common Utilities)**
Hadoop Common includes core Java libraries and scripts needed by all components in Hadoop ecosystem such as HDFS, YARN and MapReduce. These libraries are used for file system and I/O operations, configuration and logging, security and authentication and network communication. In easy way to understand, Hadoop Common provides shared libraries and utilities that help all Hadoop components work together. [8]

Although this framework renders Hadoop very scalable and reliable, it also presents some performance challenges particularly because it is based on disk operations.

B. Strengths of Apache Hadoop

The fact that Hadoop can process large volumes of data is one of the primary reasons why it has become so popular. It has a number of significant benefits:

- **Scalability:** Hadoop can be easily expanded to add more machines to the system, and it is appropriate to work with growing data volumes.
- **Fault Tolerance:** The system is replicated so that it is not affected when hardware fails.
- **Cost Effectiveness:** Hadoop can be run on comparatively cheap hardware and it is a viable solution to most organisations.
- **Parallel Processing:** Hadoop is able to process large volumes of data more efficiently than conventional systems by assigning tasks to multiple nodes.

These capabilities of Hadoop render it especially applicable to the batch processing problems, which require processing a large amount of data over time.

C. Limitations of Apache Hadoop

Despite the advantages, Hadoop has its limitation, particularly when it is used in the context of contemporary data processing requirements:

- **Disk-Based Processing:** Hadoop often stores intermediate data in disk, which is slow because of the high input or output activities[7].
- **Poor Real- Time Processing:** Hadoop is not a good fit to support real-time analytics, it is mostly a batch processing system.
- **Complex Development Process:** MapReduce programs may take time to write and may need more of an insight into the framework.
- **Inefficiency on Iterative Tasks:** Tasks that need to be repeated, like machine learning algorithms are less efficient because they need repeated disk access.

These limitations have been widely discussed in previous studies, particularly regarding Hadoop's inefficiency in handling iterative and real-time processing tasks [5],[6],[7].

III. APACHE SPARK OVERVIEW

Apache Spark came in as an improved framework that overcomes most of the shortcomings of Hadoop. In contrast to Hadoop, Spark operates data in memory, which minimizes the disk operations drastically.

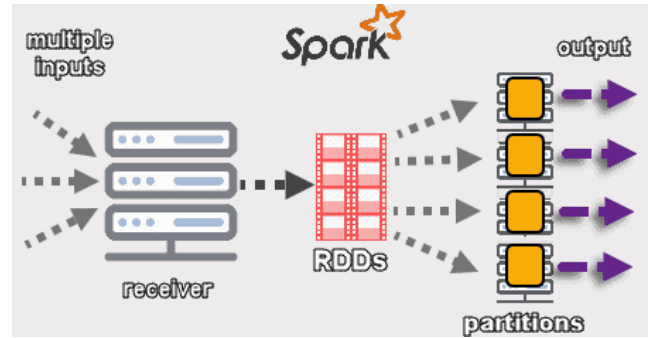


Figure 2: Apache Spark workflow diagram.

Spark operates on the concept of Resilient Distributed Datasets (RDDs) that enables data to be stored and reused in memory throughout various operations. It is especially useful in the tasks that need to be computed repeatedly[2]. Moreover, Spark is more flexible and easier to use since it supports various programming languages and has in-built tools to support advanced analytics.

IV. COMPARATIVE ANALYSIS

A. Workflow Diagram Comparison

To understand the performance gap between these two frameworks, we must analyze how data moves through their respective execution pipelines. Figure 3 provides an architectural side-by-side comparison of how hardware interactions handle multi-stage iterative tasks within each engine.

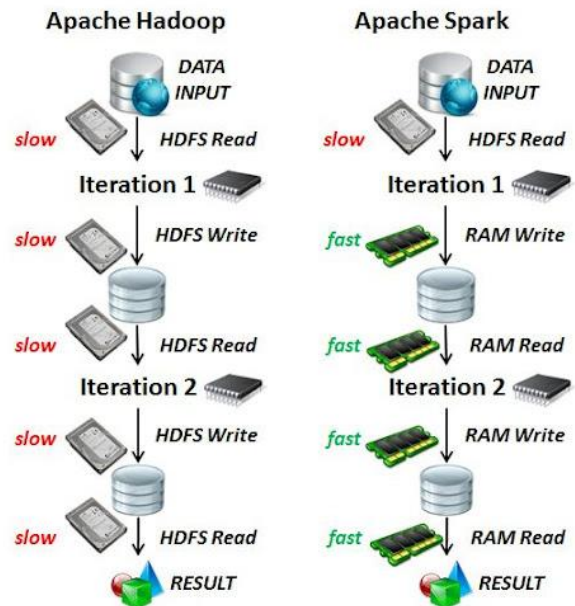


Figure 3: Comparison workflow between Hadoop and Spark

As illustrated in Figure 3, the processing pipelines differ significantly in how they handle intermediate data states across successive processing loops:

- **Apache Hadoop**
The execution pipeline is disk bound. In Iteration 1, the system reads the data from the storage tier (HDFS Read). But when the computational phase is over, the intermediate state cannot be kept in memory, it must be written completely to physical disk space through a high-latency HDFS Write operation. At the start of Iteration 2, the system is forced to go through another slow physical disk read cycle (HDFS Read). This back-and-forth of computation and physical magnetic disks leads to serious input/output (I/O) bottlenecks.
- **Apache Spark**
It is the framework that relies on a simple memory-mapped pipeline. The behavior of Spark changes completely for the rest of the lifecycle of the job, except for the first step, which is a disk read from the data input layer (HDFS Read). After Iteration 1, intermediate data states are recorded directly in high-speed volatile cluster memory (RAM Write). During Iteration 2, Spark immediately reads the dependencies from memory (RAM Read) without accessing the physical disk at all until an output is generated by a final action. This is how Spark enables iterative applications to run much faster.

B. Core Metric Comparison

When comparing Apache Hadoop and Apache Spark, it is important to note the following differences in their data processing:

Criteria	Apache Hadoop	Apache Spark
Processing Model	Disk-based execution	In-memory pipelining
Speed	Slower	Faster
Real-Time Processing	Limited	Supported
Ease of use	Complex	More flexible
Resource Requirement	Lower memory usage	Higher memory usage

By looking at the comparison table above, the disk-based nature of Hadoop may lead to slower performance, particularly with large or iterative loads. Conversely, Apache Spark can execute its tasks faster because of in-memory processing, which makes it more appropriate to the current data processing requirements[7]. This enhances performance, however, can be at the expense of increased memory usage.

C. Case-Proven Performance Analysis

To break out from theory, let's consider a real-world enterprise infrastructure deployment that illustrates the impact of these architectural differences:

Case Study A: Industrial IoT Predictive Maintenance at General Electric (GE)

General Electric (GE) has a global Industrial IoT platform, Predix, that collects and analyzes real-time sensor data telemetry from asset deployments such as commercial jet engines and power turbines [9].

The Problem: Consuming and storing petabytes of unstructured diagnostics and time-series log data without expensive hardware or risk of data corruption.

The Architecture: GE built its base big data lake with a multi-node Apache Hadoop distribution, exclusively built around the Apache Hadoop Distributed File System (HDFS) storage.

Empirical Performance: GE used the horizontal storage capacity of Hadoop, and batch-oriented MapReduce routines to perform large-scale long-term retrospective maintenance algorithms [9]. The platform can accurately predict machinery fatigue and recommend proactive maintenance by studying the machinery lifecycle for multi-year baselines. This reduces the cost of running the application for the client and helps avoid unplanned equipment failures, making Hadoop very cost effective for batch historical processing, which is non-time critical [9].

Case Study B: Content Personalization with a Focus on Latency at Netflix

Netflix has a large-scale entertainment network spanning the globe, with real-time user personalization and recommendation models being the key to user engagement and subscriber retention.

The Challenge: Handling billions of dynamic viewing interactions and making changes to user interfaces in real time. Legacy disk-bound batch processing engines caused significant computational lag and prevented personalizing the homepage in real time.

The Architecture: Netflix personalization and machine learning pipelines were moved to a high-performance Apache Spark cluster with the memory abstraction tier of Resilient Distributed Dataset (RDD).

Empirical Performance: Spark is the first to completely avoid physical disk read/write boundaries by caching intermediate algorithmic features and user action logs directly in volatile cluster memory (RAM) [10]. This memory-mapped pipeline allowed the recommendation engines to process and ingest huge numbers of behavioral updates simultaneously. This led to a significant improvement in execution times, allowing Netflix to update personalized interface arrays and recommendation indices in sub-second reaction windows.

V. DISCUSSION

Apache Spark's in-memory model represents a fundamental shift in the design paradigm of modern data engineering, compared to the MapReduce model of Apache Hadoop. Big data analytics in the early days focused on storing a lot of data and keeping costs low. For those basic needs, the horizontally scalable, disk-intensive, and cost-effective design of Hadoop was ideal. In today's business context, however, data is

considered as an asset that is instantly available for use and exploration and can be queried instantly.

Comparing the two is technically incorrect, as Spark is better at computational processing speeds than Hadoop, but these two are not mutually exclusive. Spark does not have any native long-term distributed storage tier, it's mainly a computational processing engine. As a result, the business environment has shifted towards hybrid data environments.

Modern enterprise architecture does not eliminate Hadoop but rather integrate Hadoop with it. They use Hadoop's HDFS as a low-cost storage repository to securely store multi-petabyte historical data and deploy Apache Spark atop it as an in-memory execution engine to perform real-time streaming analytics, interactive SQL queries, and distribute machine learning models over the shared data. This integrated design enables companies to maximize storage efficiency and computing power.

VI. CONCLUSION

Apache Hadoop is a steppingstone in the evolution of distributed systems and a significant achievement in the ability of commodity hardware clusters to store and process petabyte-scale data in a secure manner. Its disk-dependent MapReduce engine is very reliable for large batch operations without time constraints, but its structural I/O bottlenecks are not suitable for real-time and iterative workloads. Apache Spark overcomes these design constraints with memory-mapped pipelines and Resilient Distributed Datasets, offering a flexible and agile platform for real-time processing and sophisticated data science workloads. In the end, it comes down to assessing particular operational constraints, data

velocity needs, and infrastructure budgets when choosing between these platforms. The combination of Hadoop's low-cost storage capabilities and Spark's fast in-memory computation power is a powerful model for today's enterprise big data architectures as the amount and complexity of corporate data continues to grow.

REFERENCES

- [1] Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107–113.
- [2] Zaharia, M., et al. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65.
- [3] Shvachko, K., Kuang, H., Radia, S., & Chansler, R. (2010). The Hadoop Distributed File System. *IEEE Symposium on Mass Storage Systems*.
- [4] Vavilapalli, V. K., et al. (2013). Apache Hadoop YARN: Yet another resource negotiator. *ACM Symposium on Cloud Computing*.
- [5] Sewal, P., & Singh, H. (2021, October). A critical analysis of apache hadoop and spark for big data processing. In *2021 6th International Conference on Signal Processing, Computing and Control (ISPCC)* (pp. 308-313). IEEE.
- [6] Utami, F. D., & Astuti, F. D. (2024). Comparison of Hadoop Mapreduce and Apache Spark in Big Data Processing with Hgrid247-DE. *Journal of Applied Informatics and Computing*, 8(2), 390-399.
- [7] Ahmed, N., Barczak, A. L., Susnjak, T., & Rashid, M. A. (2020). A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. *Journal of Big Data*, 7(1), 110.
- [8] GeeksforGeeks, "Hadoop Architecture," *GeeksforGeeks*, Aug. 12, 2025. <https://www.geeksforgeeks.org/big-data/hadoop-architecture/>
- [9] Acceldata. (2024). *Hadoop Analytics: Big Data Analysis for Real-Time Insights*. Acceldata Enterprise Architecture Insights.
- [10] NetApp Instaclustr. (2023). Real-World Application Use Cases with Apache Spark: Scaling Recommendation Systems. NetApp Whitepapers.